# PlaceRight: Campus Recruitment Portal

A full-stack placement portal (React, FastAPI, PostgreSQL) that enforces eligibility, a status workflow, offer limits and clash-free interviews, and stays correct under retries and races.

**Kriti Singhal** · Technical documentation · October 2026

GitHub: github.com/krit566/campus-recruitment-api

# Contents

1. Overview

2. Features

3. Architecture

4. Data model

5. Rules the system enforces

6. Walkthrough: real requests

7. Concurrency and retries

8. Security

9. Web app

10. Testing and delivery

11. Limitations and next steps

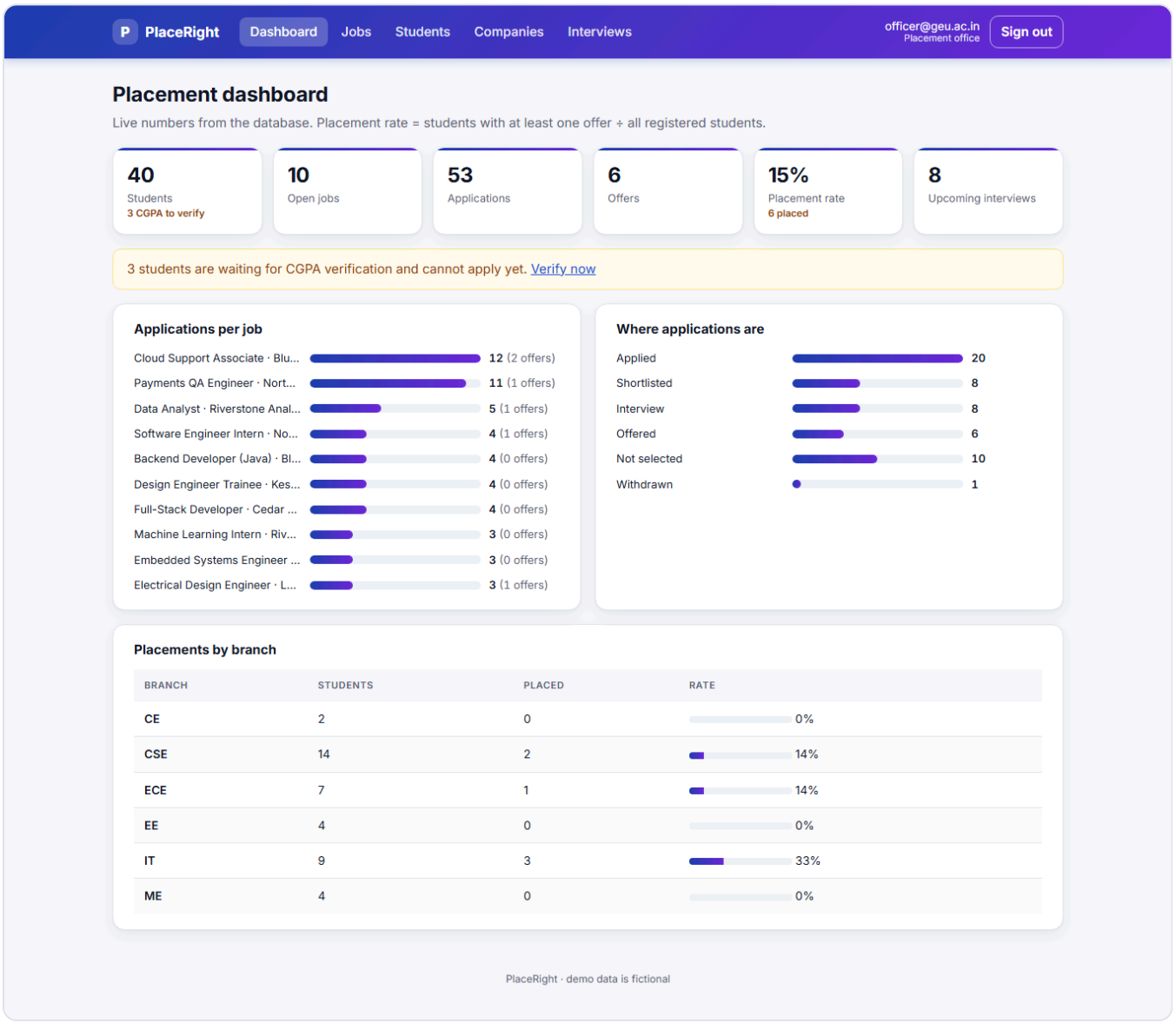
12. Interview questions

# 1 Overview

**PlaceRight** is a full-stack campus placement portal. Students sign up with their college email, see only the jobs they are eligible for, apply once and follow each application from "applied" to "offered". The placement office verifies CGPAs, posts jobs, moves applicants through a kanban pipeline, books interview slots without clashes and exports applicant lists. Beyond create/read/update, it stays correct when things go wrong: **clients retry, two reviewers click at the same moment, two officers book the same panel, a student tries to act as someone else**.

| | | | |
|---|---|---|---|
| **77** backend tests against real PostgreSQL | **11** web-app tests incl. axe accessibility checks | **1 of 8** parallel offers wins when one opening is left | **2** roles: student and placement office |

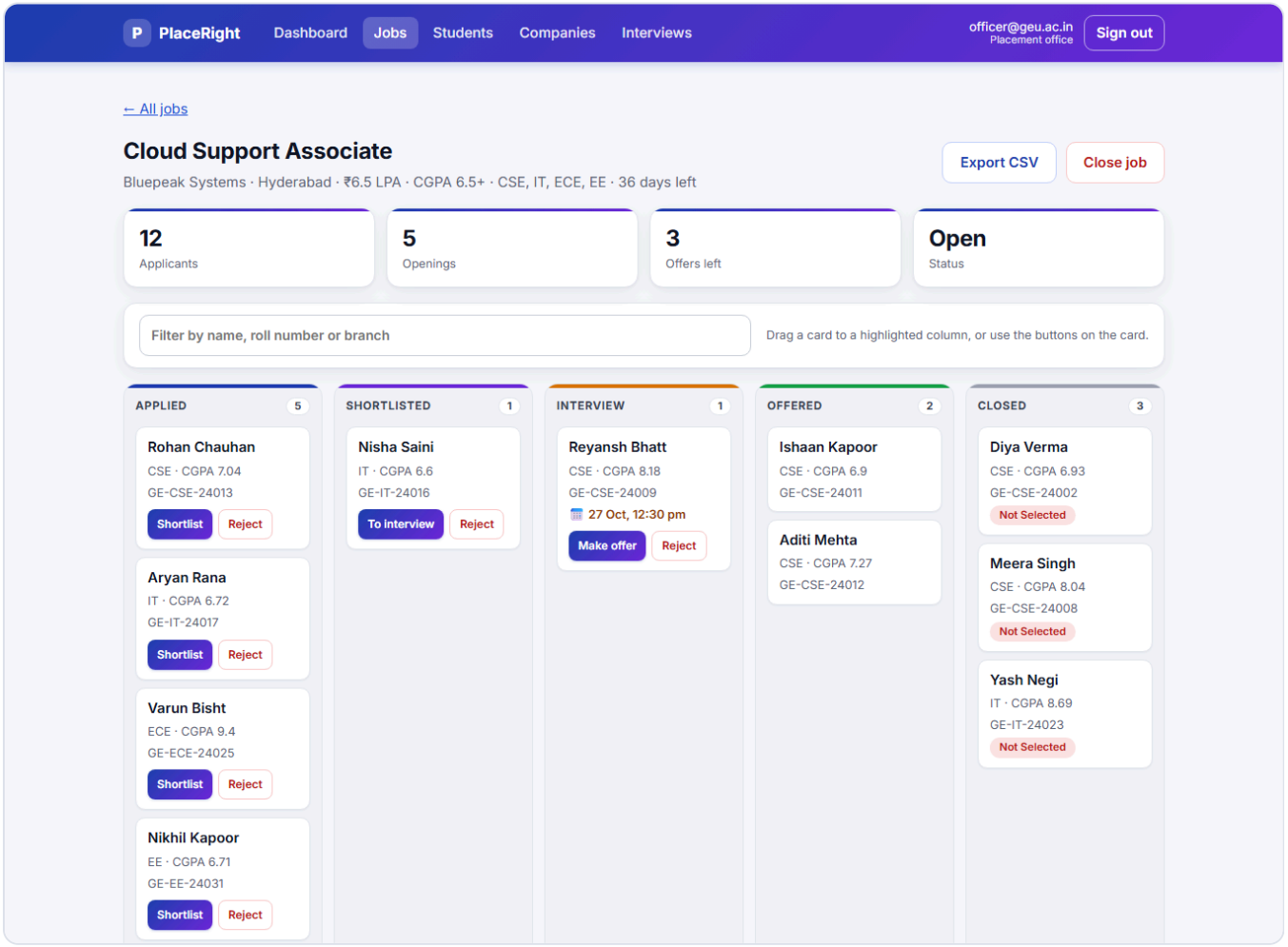
React + TypeScript (Vite, React Router) · FastAPI · PostgreSQL with plain SQL and Alembic migrations · JWT + bcrypt · Docker Compose (Postgres, API, nginx) · GitHub Actions · Render blueprint



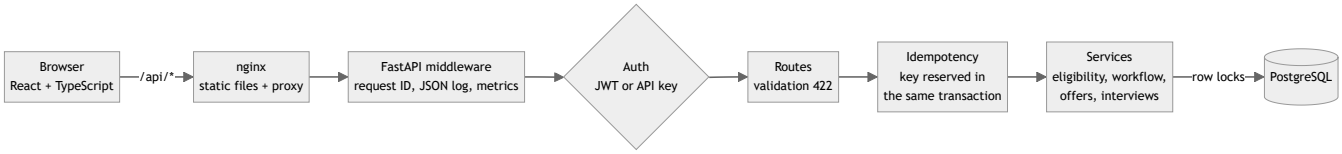
> All people and companies in the screenshots are fictional demo data created by `seed.py`. The numbers on the dashboard are whatever is in the database; no real usage is claimed.

## 2 Features

| Student | Placement office |
|---|---|
| Sign up with a college email (other domains refused); sign in | Dashboard: students, open jobs, applications, offers, placement rate, upcoming interviews |
| Jobs filtered by branch and verified CGPA, with the reason a job is not open | Charts: applications per job, where applications are, placements by branch |
| One-click apply, safe to retry | Per-job kanban pipeline; only valid moves offered (buttons or drag and drop) |
| Progress bar and timeline for every application; withdraw before the interview | Applicant detail: profile, resume, audit history, interview scheduling |
| Resume upload (PDF, max 2 MB) | CGPA verification, companies, jobs (create, close), CSV export |
| Notifications for status changes and interview invitations | Interview calendar grouped by day |

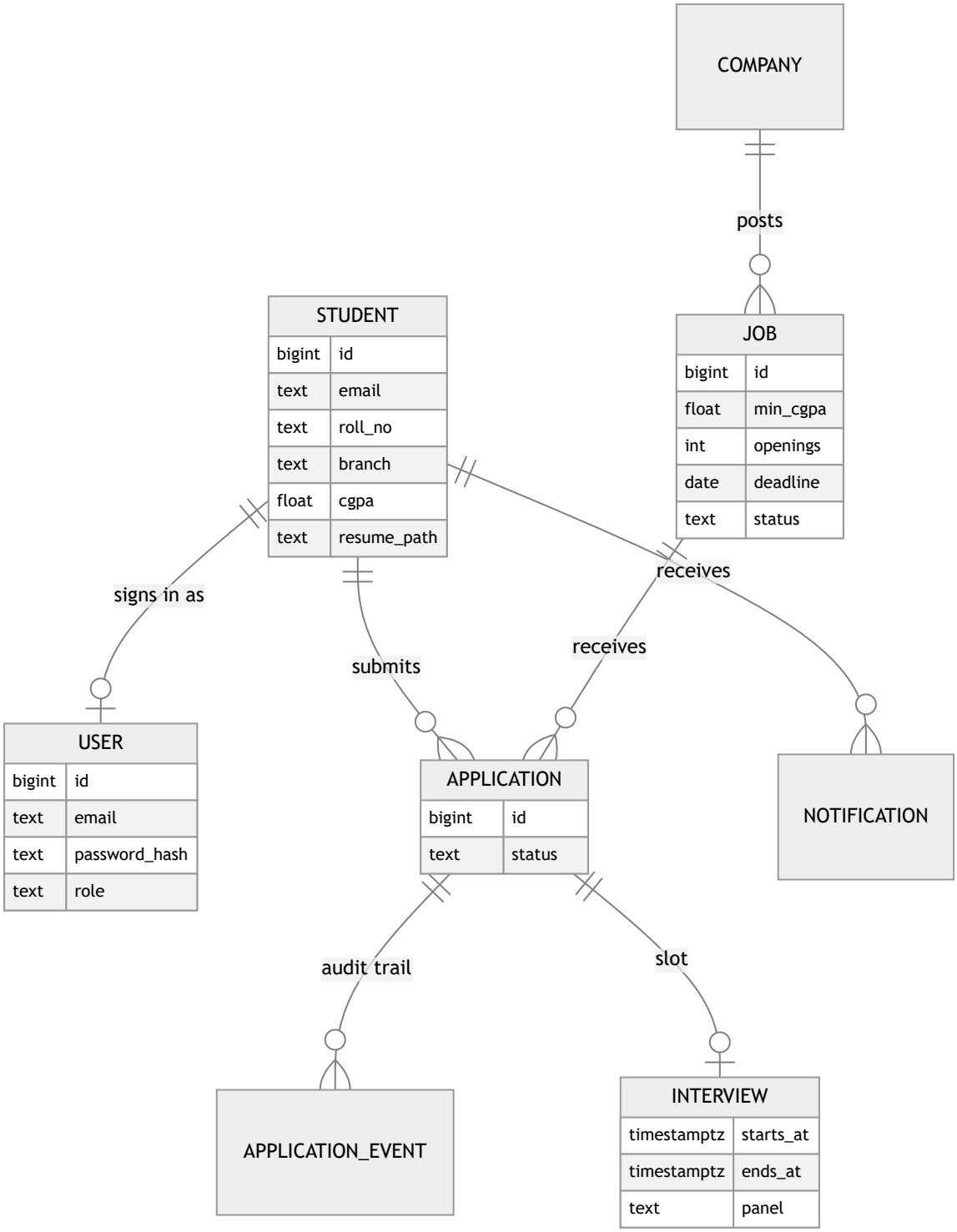


## 3 Architecture




| Layer | File | Knows about |
|---|---|---|
| Web app | `frontend/src` | Pages, forms, the API client ( `api.ts` ), who is signed in ( `auth.tsx` ) |
| HTTP | `backend/app/main.py` | Routes, status codes, who may call each route |
| Validation | `schemas.py` | Email, CGPA 0–10, branch codes, time zones, lengths |
| Business rules | `services.py` | Eligibility, workflow, offer limit, interview clashes, notifications |
| Data | `repository.py` | SQL only (every value is a query parameter) |
| Cross-cutting | `security.py` , `idempotency.py` , `uploads.py` , `observability.py` | Passwords and tokens, retry safety, files, request IDs and metrics |

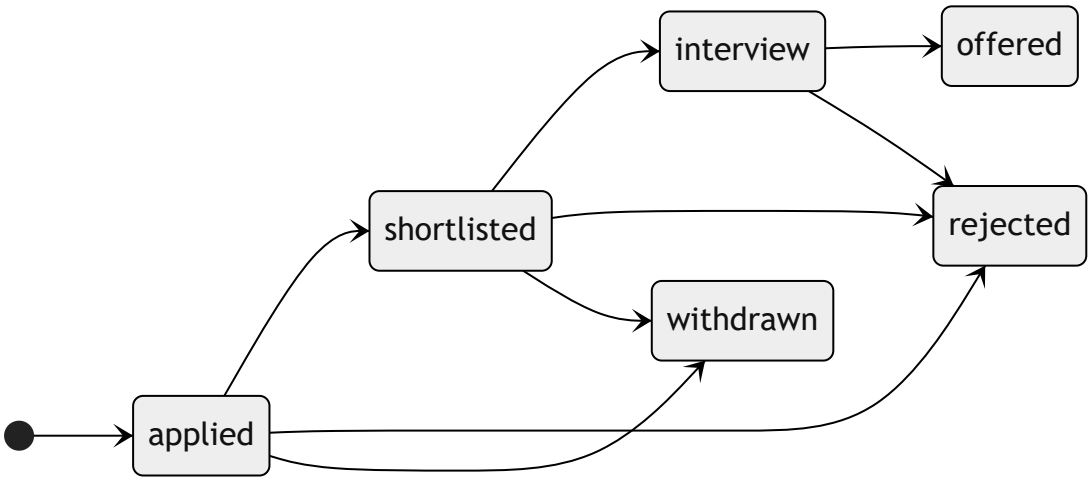
## 4 Data model




- `students.cgpa` is `NULL` until the placement office enters it; nothing is open to a student before that.
- `UNIQUE (student_id, job_id)` on applications: one application per job, enforced by the database even under parallel requests.
- Schema changes are Alembic migrations (`backend/migrations`), applied on start under a Postgres advisory lock so two instances never migrate at once.

## 5 Rules the system enforces

| Rule | When broken |
|---|---|
| Valid input: college email for sign-up, CGPA 0–10, known branch, openings > 0, future deadline, interview times with a time zone | 422 |
| Unique email, roll number, company; one application per student per job | 409 |
| Branch eligible; CGPA verified and at least the minimum | 403 |
| Students only see and change their own records; cannot set CGPA; can only withdraw | 403 |
| Status moves only along the workflow; no more offers than openings (job closes itself when full) | 409 |
| No overlapping interviews for one student or one panel | 409 |
| Resume is really a PDF (first bytes) and at most 2 MB | 415 / 413 |
| Five wrong passwords for one email in 15 minutes | 429 |




## 6 Walkthrough: real requests

Output of `scripts/walkthrough.py` against a fresh local server (one job with **one** opening, minimum CGPA 7.5, CSE/IT only):

```
Sign-up with a non-college email
  -> 422  {"detail": "use your college email address (@geu.ac.in)"}
Asha applies before her CGPA is verified
  -> 403  {"detail": "your CGPA has not been verified by the placement office yet"}
Asha applies, Idempotency-Key: a1
  -> 201  {"id": 1, "status": "applied"}
same request retried after a timeout (key a1)
  -> 201  Idempotent-Replayed: true  {"id": 1, "status": "applied"}
Meera (ME branch) applies
  -> 403  {"detail": "branch ME is not eligible for this job"}
Kabir (CGPA 6.9) applies
  -> 403  {"detail": "CGPA 6.9 is below the minimum 7.5"}
Ravi tries to apply as Asha
  -> 403  {"detail": "students can only access their own records"}
Ravi tries to set his own CGPA
  -> 403  {"detail": "CGPA is updated by the placement office from official results"}
Officer: applied -> offered (skips the interview)
  -> 409  {"detail": "cannot move an application from 'applied' to 'offered'"}
Interview for Ravi, same panel, 15 min after Asha's
  -> 409  {"detail": "panel 'Panel A' is already interviewing from 27 Oct 2026, 10:30 AM IST to
11:00 AM"}
Two reviewers offer the last opening at the same moment
  -> [200, 409]  (exactly one 200, the other 409)
GET /jobs/1  -> {"status": "closed"}
```

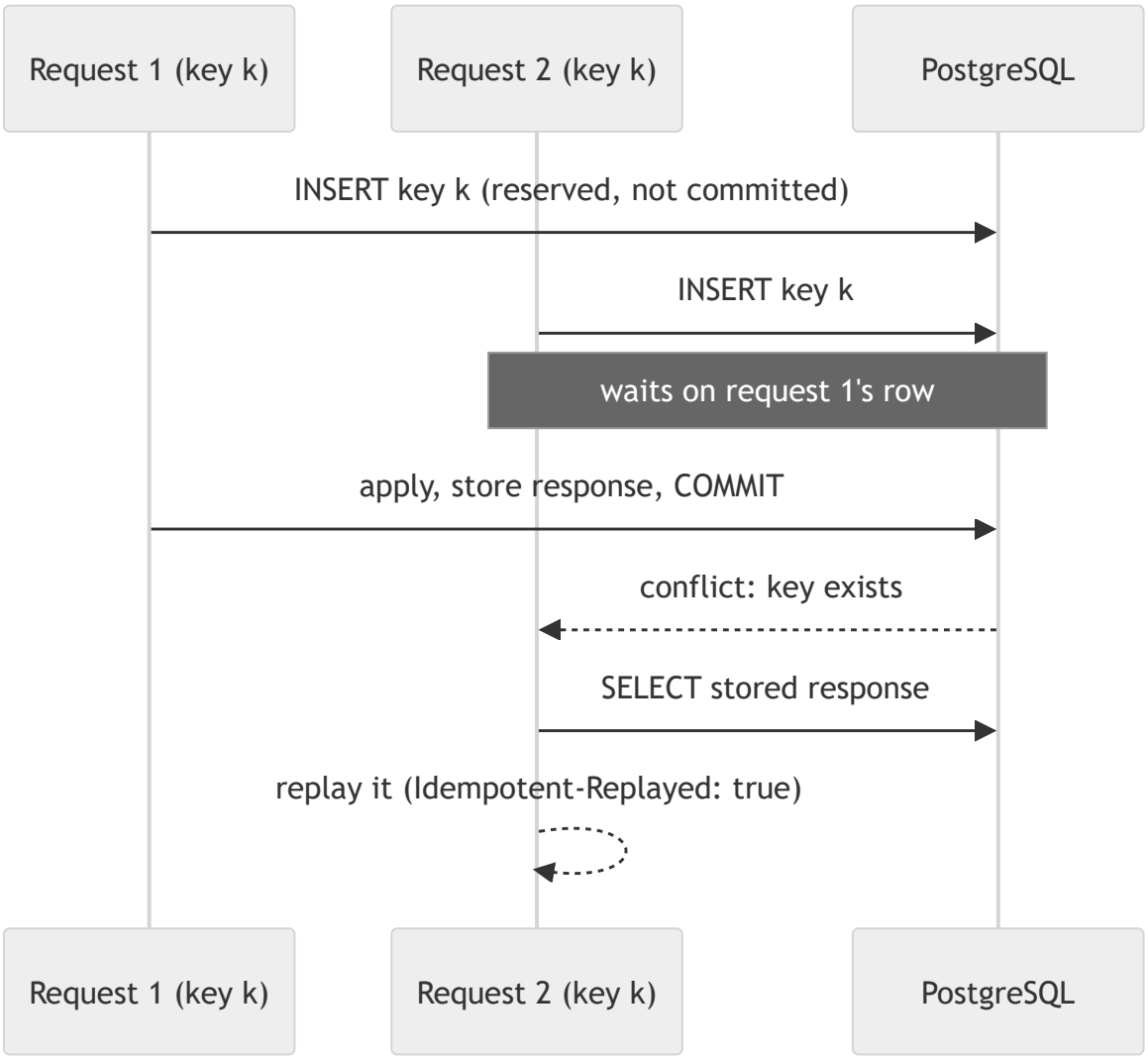
## 7 Concurrency and retries

### The last opening

`change_status` locks the job row with `SELECT ... FOR UPDATE` before reading anything, then locks the application. A second reviewer's transaction waits on the job lock, then sees the first offer and gets 409. The update is also compare-and-set ( `WHERE status = <status we read>` ). Locks are always taken job first, then application, so two transactions can never wait on each other in a circle (no deadlock). Applying takes a `FOR SHARE` lock, so many students can apply at once but a job cannot be closed in the middle of an application.

### Idempotency keys

The first version (SQLite) stored the key *after* the action, so two simultaneous first attempts with the same key could both run. Now the key is **reserved first, in the same transaction**:




If request 1 fails, its reservation rolls back with it and request 2 runs normally, so a corrected retry works. A test sends 8 simultaneous first attempts with one key: one application is created and 7 responses are replays.

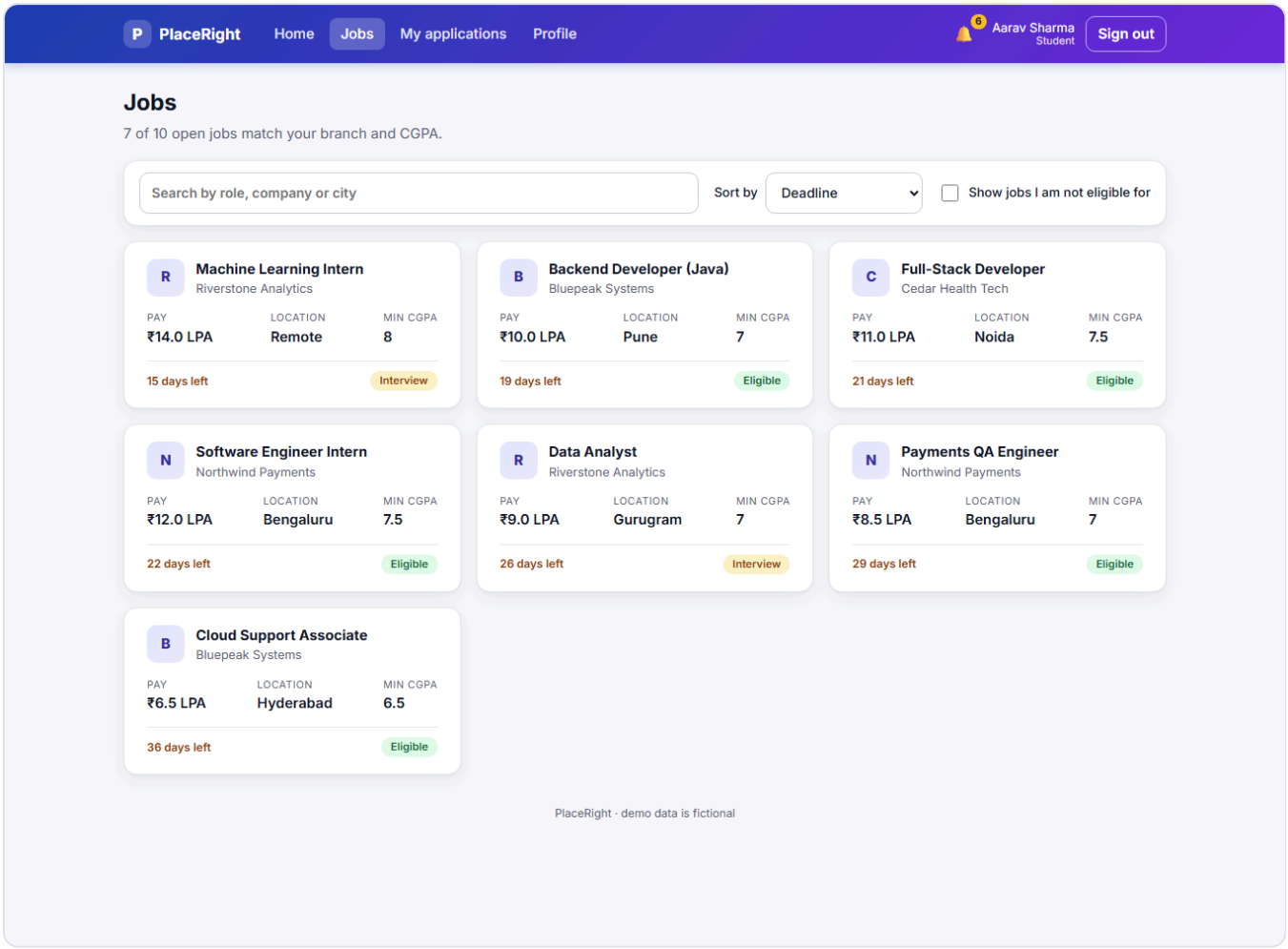
### Interview clashes

"Check for an overlap, then insert" is a race. `schedule_interview` locks the student's row and takes a per-panel advisory lock ( `pg_advisory_xact_lock(hashtext(lower(panel)))` ), in that order, before checking and writing. A test books one panel for six students in parallel: exactly one succeeds. (A Postgres exclusion constraint on time ranges would be the database-level alternative; it needs the `btree_gist` extension.)

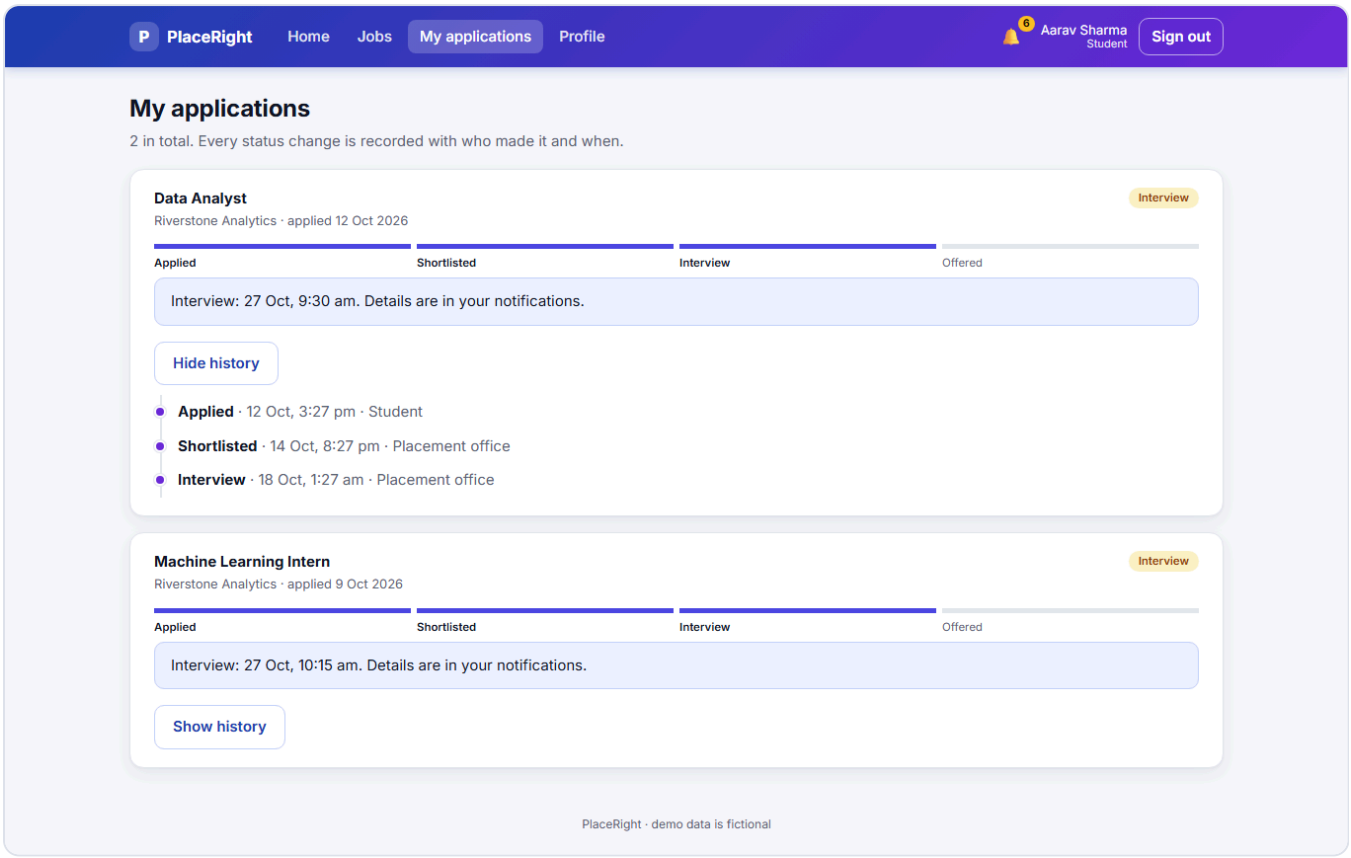
## 8 Security

| Concern | How |
|---|---|
| Passwords | bcrypt (cost 12). An unknown email is checked against a dummy hash, so response times do not reveal which emails have accounts; the error message is the same. |
| Sessions | HS256 JWT, 8-hour expiry, algorithm list fixed on decode; startup warns if `JWT_SECRET` is missing or shorter than 32 bytes. |
| Brute force | 5 wrong passwords for one email in 15 minutes → 429. |
| Authorisation | Checked on every route and again in services; students only reach their own records. |
| API keys | For scripts: stored as SHA-256, shown once; the officer key is compared in constant time. |
| SQL injection | Every value is a query parameter; column names for updates come from a fixed allow-list. |
| Uploads | Type from the file's bytes, size limit while reading, random stored names (no path traversal). |
| CSV injection | Cells starting with `= + - @` are prefixed with `'`. |
| Headers | `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`; nginx adds `X-Frame-Options: DENY`. |

## 9 Web app




- One API client (`api.ts`) adds the token, turns error responses into readable messages and signs the user out when a session expires.
- Each "Apply" or status change gets an Idempotency-Key; retrying the same action reuses it.
- The kanban asks the API which moves are allowed (`/meta/workflow`) and shows only those; the API still checks every move.
- Accessibility: labelled fields, focus moved to the page heading after navigation, a skip link, live-region announcements on the board, keyboard-usable buttons for every drag-and-drop action, AA-contrast status colours; axe-core runs in the tests.
- Responsive down to 375 px: the kanban columns stack and wide tables scroll inside their card, so the page never scrolls sideways.




## 10 Testing and delivery

| Suite | Covers |
|---|---|
| `test_students.py`, `test_jobs_and_applications.py` | Validation, uniqueness, filters, eligibility, workflow, offer limit, auto-close, stats |
| `test_security_reliability.py` | API keys, roles, idempotent replay and conflicts, audit trail, request IDs, metrics |
| `test_auth.py` | Sign-up rules, bcrypt storage, login, lockout, expired and forged tokens, CGPA verification, first-officer bootstrap |
| `test_uploads.py` | PDF check, size, path traversal, replacement, access control |
| `test_interviews_notifications_export.py` | Interview rules and clashes, notifications, CSV export, dashboard numbers |
| `test_concurrency.py` | 8 parallel offers (1 wins), 8 simultaneous first attempts with one key (1 application), 6 parallel bookings of one panel (1 wins) |
| `frontend/src/test` | Sign-in and role routing, expired sessions, eligibility reasons, idempotent apply retry, pipeline moves, axe checks |

Backend tests run against a real PostgreSQL: a service container in CI, or a throwaway server started with `pgserver` locally, because the guarantees above are database behaviour a fake would not reproduce. The concurrency tests passed 31 runs in a row while this was built. CI also runs the seed script, type-checks and builds the web app, and builds both Docker images.

```
docker compose up --build            # http://localhost:8080
docker compose exec backend python seed.py
cd backend; python -m pytest -q       # 77 tests
cd frontend; npm test                 # 11 tests
```

## 11 Limitations and next steps

- The token lives in `sessionStorage`: simple, but readable by any XSS. An HttpOnly cookie with CSRF protection is stronger.
- Login lockout and metrics are in memory (per instance); several instances would use Redis or the database, and Prometheus.
- No email verification, password reset or refresh tokens yet.
- Resumes are on a local volume; Render's free tier has no persistent disk, so object storage is the real fix.
- Notifications are in-app only.
- Demo data is generated and fictional; there are no real usage figures.

## 12 Interview questions

**Why lock the job row and not just check the offer count?**
Two transactions can both read "1 opening left" before either writes. `FOR UPDATE` makes the second wait until the first commits, so it reads the new count.

**Why reserve the idempotency key before doing the work?**
Inserting the key first makes Postgres serialise requests with the same key: the second waits on the first's uncommitted row, then replays its response. Storing it afterwards leaves a window where both run.

**Why is a UNIQUE constraint still needed?**
Application code checks can be raced; the database enforces the rule at write time no matter how requests interleave.

**Why bcrypt for passwords but SHA-256 for API keys?**
Passwords are short and guessable, so hashing must be slow. API keys are long random strings, so a fast hash is enough.

**What stops a forged token?**
The signature is checked with the server's secret and only HS256 is accepted, so a token signed with another key (or with "none") is rejected.

**401 vs 403 vs 409 vs 422?**
Not signed in; signed in but not allowed; conflicts with the current state; the request itself is invalid.